

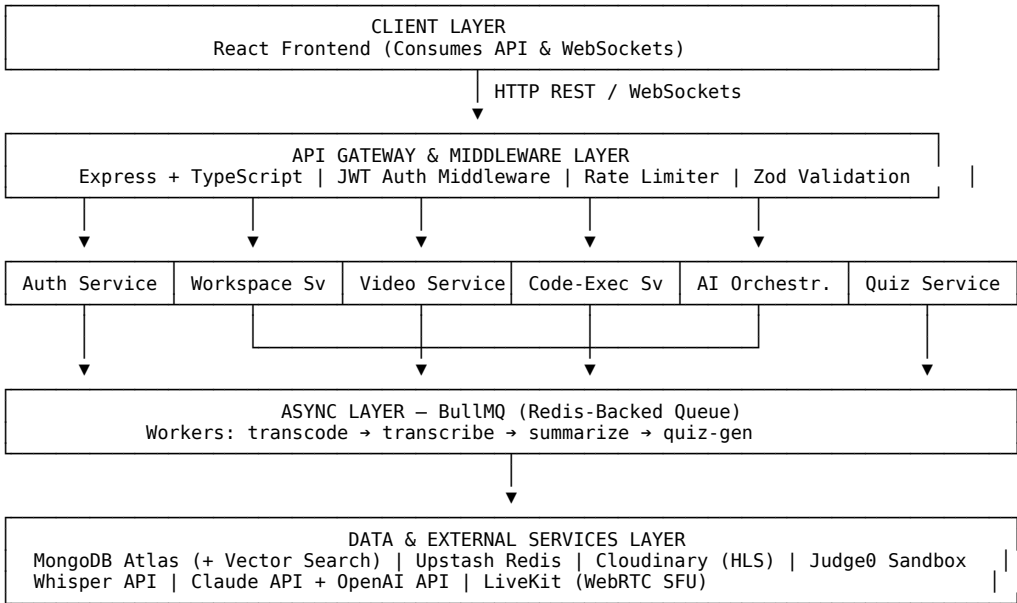
Backend Architecture & Technical Blueprint

Document 1: Backend Architecture & Technical Blueprint

1. Executive System Architecture & System Design

The backend for the **Smart Learning & AI Media OS** is designed as a **Modular Monolith** built with Node.js, Express, and TypeScript within a `pnpm` monorepo`` structure located at `apps/api/src/``. Person A (Backend Lead) owns the database, authentication, API routes, async queue processing, DevOps deployments, and code-execution sandboxes.

The architecture separates synchronous API handling from heavy, asynchronous computing operations. Synchronous REST and WebSocket traffic is processed through an API Gateway protected by JWT authentication, role-based authorization, rate limiting, and Zod runtime schema validation. Heavy background tasks—such as video audio extraction, speech-to-text transcription, vector embedding generation, LLM summarization, and sandboxed code execution—are offloaded to a Redis-backed BullMQ job queue.



2. Monorepo Directory Structure & Service Modules

All backend code resides in `apps/api/src/`` inside the monorepo:

```
apps/api/src/
├── config/           # DB connections, Redis configuration, Zod env validation
├── middleware/       # Auth, RBAC, Rate limiting, Request validation, Error handling
├── models/          # Mongoose schemas & TypeScript interfaces
└── services/        # Business domain logic
```

auth/	# Login, Register, JWT rotation, Bcrypt password hashing
workspace/	# Dual-workspace context management
video/	# Video metadata, Cloudinary HLS triggers, state transitions
code-exec/	# Judge0 execution proxy & sandbox constraints
ai/	# Unified AI Router (Claude + OpenAI + Whisper API)
quiz/	# Quiz generation, scoring logic, attempt persistence
workers/	# BullMQ job processors & handlers
routes/	# Express route controllers
index.ts	# Entry point & HTTP/WebSocket server setup

Detailed Module Responsibilities

1. **API Gateway & Middleware Layer** (`apps/api/src/middleware/`)**:

* `auth.ts`: Verifies incoming Bearer access tokens.

* `rateLimiter.ts`: Uses Upstash Redis sliding-window algorithms to rate-limit AI and execution endpoints.

* `validate.ts`: Enforces runtime Zod validation schemas on incoming `req.body`, `req.query`, and `req.params`.

* `error.ts`: Standardizes JSON error outputs and captures unhandled exceptions in Sentry.

2. **Auth Service** (`apps/api/src/services/auth/`)**:

* Hashes user passwords using `bcryptjs`.

* Issues short-lived access tokens (15-minute expiration) and long-lived refresh tokens (7-day expiration in `httpOnly` cookies).

* Implements Role-Based Access Control (RBAC) across three explicit roles: `student`, `instructor`, and `org\_admin`.

3. **Workspace Service** (`apps/api/src/services/workspace/`)**:

* Manages the **Dual Workspace Engine**:

* **Private Personal Workspace**: Automatically instantiated upon registration, strictly isolated to the user ID.

* **Shared Organizational Workspace**: Tenant-scoped to a college, company, or institutional domain (`currentOrgId`).

- * Handles workspace context switching (``/api/v1/workspaces/switch``) and domain-restricted access controls.

4. **Video Service** (``apps/api/src/services/video/``):

- * Receives video upload metadata and Cloudinary HLS streaming URLs (``m3u8``).

- * Creates initial ``Video`` database records with state ``processing``.

- * Enqueues video processing jobs to the BullMQ Redis queue.

5. **Code Execution Service** (``apps/api/src/services/code-exec/``):

- * Proxies execution requests to a self-hosted or managed Judge0 Docker container REST API.

- * Enforces CPU execution limits (5-second timeout) and memory caps (128MB ceiling).

- * Supports sandboxed execution for JavaScript and Python.

6. **AI Orchestration Service** (``apps/api/src/services/ai/``):

- * Houses an **Internal LLM Router** that directs requests across Claude 3.5 Sonnet, OpenAI GPT-4o, and Whisper API.

- * Executes Retrieval-Augmented Generation (RAG) by embedding incoming queries via OpenAI ``text-embedding-3-small`` and running vector search against MongoDB Atlas Vector Search.

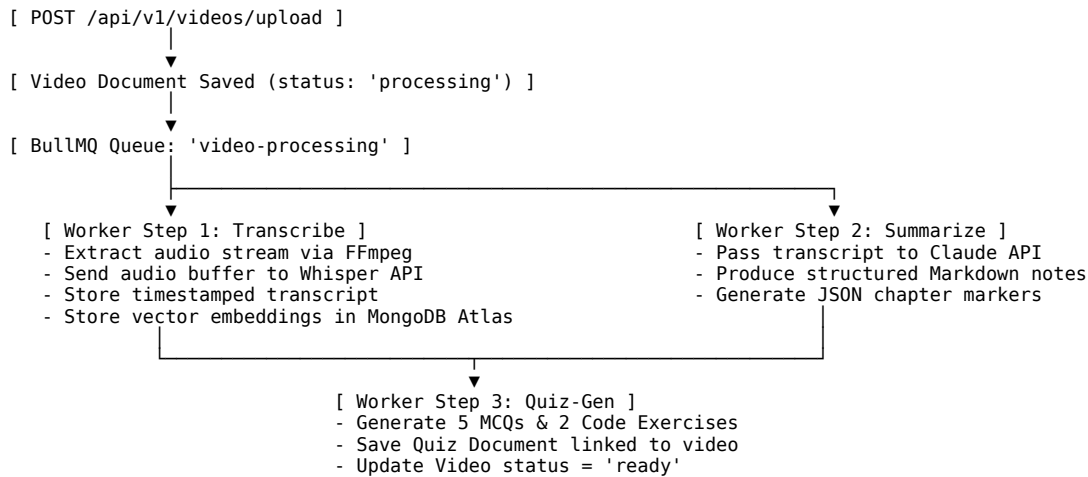
7. **Quiz Service** (``apps/api/src/services/quiz/``):

- * Persists AI-generated multiple-choice questions (MCQs), fill-in-the-blank items, and practical code challenges.

- * Grades student attempts, tracks submission scores, and updates individual/cohort analytics.

3. Asynchronous Task Queue & Worker Processing Pipelines

Long-running processing tasks run in background worker threads powered by **BullMQ** and **Upstash Redis**.



Worker Step Execution Logic

1. **Audio Extraction & Transcription (Whisper API)**:

- * Worker extracts the audio track from the uploaded video stream via FFmpeg.
- * Calls the OpenAI Whisper API to output JSON timestamped transcript segments.

2. **Vector Embedding & Storage (MongoDB Atlas)**:

- * Chunks transcript text into logical blocks.
- * Generates 1536-dimensional vector embeddings using OpenAI `text-embedding-3-small`.
- * Writes embeddings and timestamps to the `TranscriptSegment` collection.

3. **Structured Note & Chapter Summarization (Claude API)**:

- * Transmit transcript context to the Claude API.
- * Generates structured Markdown notes, executive takeaways, and JSON chapter markers.

4. **Quiz & Exercise Generation**:

* Prompts Claude API using strict JSON mode to produce MCQs and code challenges from lecture content.

* Updates `Video` document status to `ready`.

4. Complete Database Schemas & Data Contracts (MongoDB Atlas)

User & Workspace Schemas (`apps/api/src/models/`)

```
// User.ts
import mongoose from 'mongoose';

export const UserSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  email: { type: String, required: true, unique: true, index: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['student', 'instructor', 'org_admin'], default: 'student' },
  personalWorkspaceId: { type: mongoose.Schema.Types.ObjectId, ref: 'Workspace' },
  currentOrgId: { type: mongoose.Schema.Types.ObjectId, ref: 'Organization', default: null },
  createdAt: { type: Date, default: Date.now }
});

// Workspace.ts
export const WorkspaceSchema = new mongoose.Schema({
  name: { type: String, required: true },
  type: { type: String, enum: ['PERSONAL', 'ORGANIZATIONAL'], required: true },
  ownerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  organizationDomain: { type: String, default: null },
  members: [{
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    role: { type: String, enum: ['ADMIN', 'INSTRUCTOR', 'MEMBER'], default: 'MEMBER' }
  }],
  createdAt: { type: Date, default: Date.now }
});
```

Video & Vector Transcript Schemas

```
// Video.ts
export const VideoSchema = new mongoose.Schema({
  title: { type: String, required: true },
  cloudinaryUrl: { type: String, required: true },
  hlsManifestUrl: { type: String },
  duration: { type: Number, default: 0 },
  workspaceId: { type: mongoose.Schema.Types.ObjectId, ref: 'Workspace', required: true, index: true },
  uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  status: { type: String, enum: ['processing', 'ready', 'failed'], default: 'processing' },
  summaryMarkdown: { type: String, default: '' },
  chapters: [{ timestamp: Number, title: String }],
  createdAt: { type: Date, default: Date.now }
});

// TranscriptSegment.ts
export const TranscriptSegmentSchema = new mongoose.Schema({
  videoId: { type: mongoose.Schema.Types.ObjectId, ref: 'Video', required: true, index: true },
  startTime: { type: Number, required: true },
  endTime: { type: Number, required: true },
  text: { type: String, required: true },
  embedding: { type: [Number], required: true } // 1536 dimensions
});
```

Career Feature Schemas

```
// JamSession.ts
export const JamSessionSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  workspaceId: { type: mongoose.Schema.Types.ObjectId, ref: 'Workspace', required: true },
  topicPrompt: { type: String, required: true },
```

```

recordingUrl: { type: String, required: true },
transcriptText: { type: String },
metrics: {
  wordsPerMinute: { type: Number },
  fillerWordCount: { type: Number },
  clarityScore: { type: Number },
  technicalAccuracyScore: { type: Number }
},
aiFeedbackSummary: { type: String },
createdAt: { type: Date, default: Date.now }
});

// TargetResume.ts
export const TargetResumeSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  targetCompany: { type: String, required: true },
  targetRole: { type: String, required: true },
  atsScore: { type: Number, required: true },
  tailoredContentJSON: { type: Object, required: true },
  missingSkills: [{ type: String }],
  createdAt: { type: Date, default: Date.now }
});

```

MongoDB Atlas Vector Search Index Configuration

Index definition configured on the `transcriptsegments` collection:

```

{
  "fields": [
    {
      "type": "vector",
      "path": "embedding",
      "numDimensions": 1536,
      "similarity": "cosine"
    },
    {
      "type": "filter",
      "path": "videoId"
    }
  ]
}

```

5. Third-Party Integration Contracts & Operational Limits

| Provider / Tool | Technical Purpose | Implementation & API Scope | Constraints & Limits |

| :--- | :--- | :--- | :--- |

| **Cloudinary** | Video hosting & streaming[cite: 1] | Ingests raw video, generates adaptive HLS manifest URLs (.m3u8)[cite: 1]. | Chunked upload mode; automatic retry on 5xx status codes[cite: 1]. |

| **Judge0** | Sandboxed code runner[cite: 1] | REST API proxy executing Python and JavaScript source code[cite: 1]. | 5s max CPU timeout; 128MB RAM limit per request[cite: 1]. |

| **Whisper API** | Speech-to-text transcription[cite: 1] | Ingests audio streams, outputs WebVTT/JSON with exact timestamps[cite: 1]. | Executed strictly in asynchronous BullMQ workers[cite: 1]. |

| **Claude API** | LLM summarization & analysis[cite: 1] | Generates notes, dynamic mind maps, quizzes, resume optimization, and JAM feedback[cite: 1]. | Uses strict JSON mode; rate limits managed via Upstash[cite: 1]. |

| **LiveKit** | WebRTC SFU[cite: 1] | Mints short-lived JWT access tokens for real-time video study rooms[cite: 1]. | Token generation completed synchronously (< 50ms)[cite: 1]. |

| **Upstash Redis** | Message broker & rate limiter[cite: 1] | Manages BullMQ job state queues and sliding-window rate limit counters[cite: 1]. | Connection pooling with automatic reconnection backoff[cite: 1]. |